Agentic Platform

Plataforma Kubernetes elástica para una flota de agentes de IA

Informe ejecutivo · Noviembre de 2026 · Repositorio Andresm98/03-agentic-platform-swe

Resumen ejecutivo

El proyecto construye, sobre **K3s autogestionado en AWS EC2**, una plataforma efímera y reproducible que ejecuta una flota de agentes de IA. Se levanta desde cero con GitHub Actions, se opera únicamente desde el navegador y se destruye al terminar. El objetivo es demostrar replicabilidad, elasticidad y portabilidad con un coste mínimo.

Resultado: la plataforma se desplegó desde cero en AWS sin pasos manuales y funcionó de extremo a extremo: GitOps, autoscaling con servidores reales, modelo de IA real, observabilidad y publicación de aplicaciones de estudiantes por Pull Request. Al cierre, los recursos de cómputo del laboratorio están **destruidos**, y la plataforma se recrea con una sola ejecución de GitHub Actions.

22	**1 → 2**	**23 s**	**0**
aplicaciones sanas por GitOps	workers con EC2 real, y vuelta a 1 sin carga	tarea completa de 3 agentes	tokens reales gastados en carga y ataque

Qué se ha demostrado

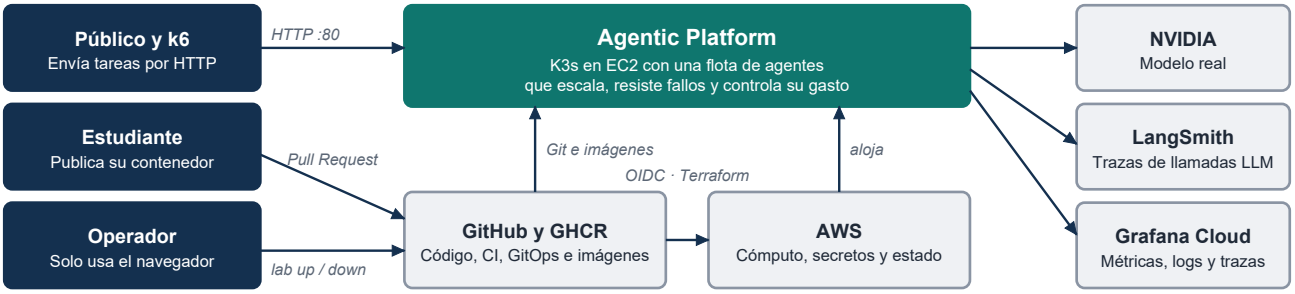
- **Reproducibilidad:** 59 recursos de AWS creados desde Terraform, y 22 aplicaciones desplegadas por Argo CD, sin intervención manual.
- **Elasticidad en dos niveles:** KEDA escala agentes desde cero; el Cluster Autoscaler añadió un EC2 real, registrado en el balanceador, y lo retiró solo en unos 9 minutos.
- **Resiliencia y control de gasto:** una ráfaga recibe *429*, y sin token de demo nunca se llega al modelo real.
- **Portabilidad del modelo:** se cambió de OpenAI a NVIDIA solo con configuración, sin tocar los agentes.
- **Autoservicio:** un estudiante publica su contenedor con un Pull Request, validado, escaneado y desplegado automáticamente.

Puntos a tener en cuenta

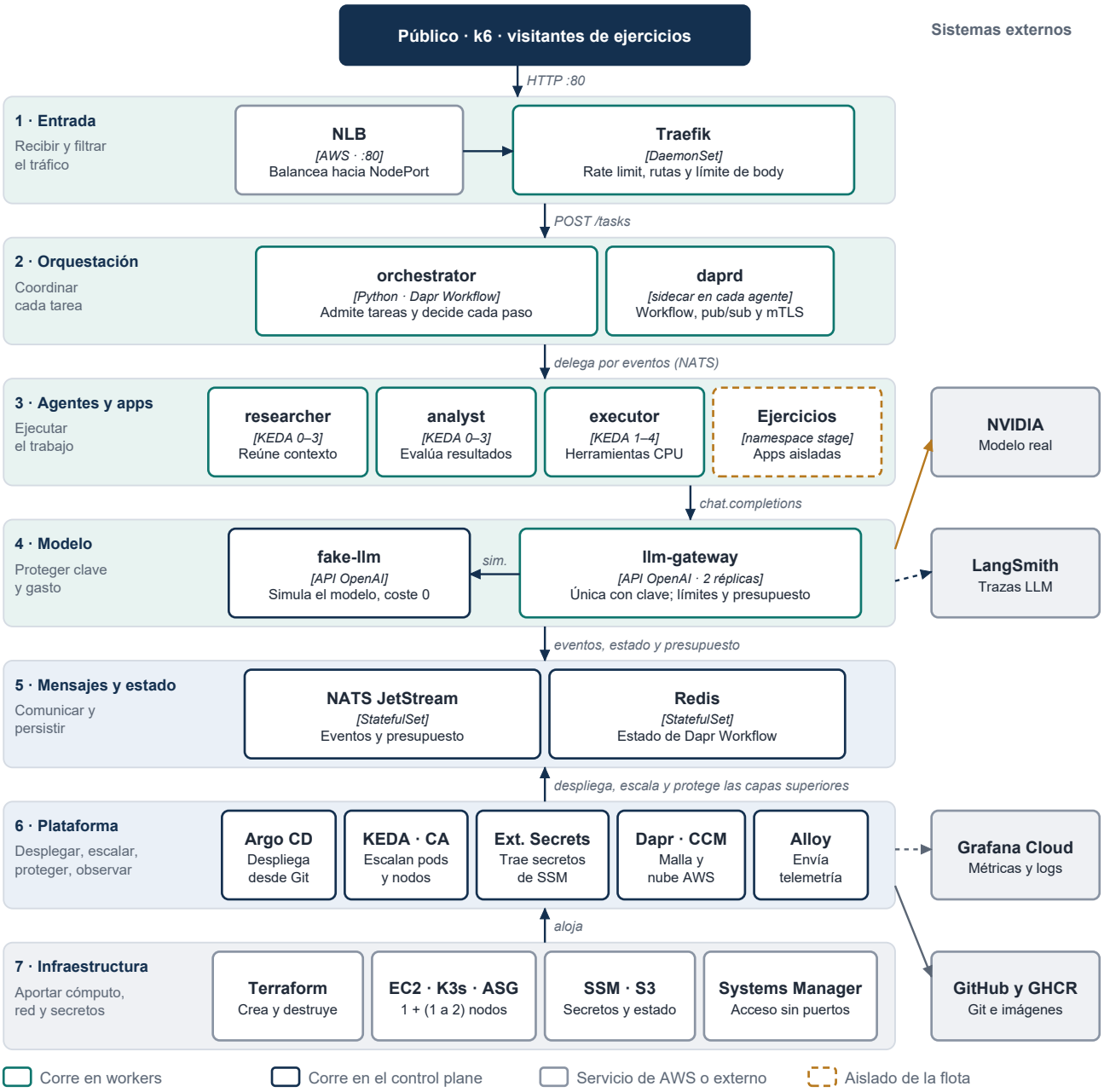
- **No es alta disponibilidad:** hay un único control plane, por decisión de coste. Si cae, el cluster se recrea desde Git.
- **Los agentes son carga de demostración:** no analizan la infraestructura en tiempo real. El producto evaluado es la plataforma.
- **Capacidad acotada a 3 EC2:** una carga sostenida puede saturar los workers. El caso se detectó y se corrigió el límite que lo agravaba.

Arquitectura (modelo C4)

Nivel 1 · Contexto. Quién usa la plataforma y de qué servicios depende.



Nivel 2 · Contenedores por capas. Cada capa tiene una responsabilidad única. Las llamadas a servicios externos salen de la capa que las necesita.

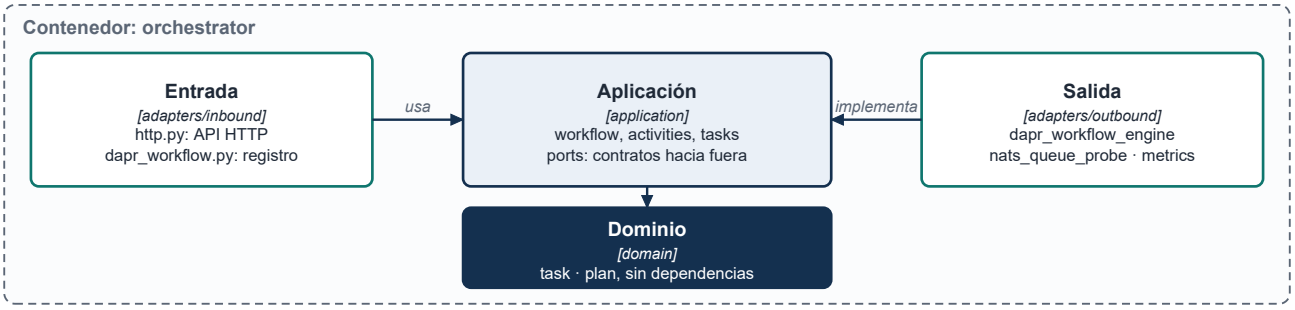


División de responsabilidades

Capa	Responsabilidad	Componentes	Lo que no hace
1 · Entrada	Recibir tráfico, limitar abusos y enrutar	NLB, Traefik	No conoce la lógica de las tareas
2 · Orquestación	Admitir tareas, pedir al modelo el siguiente paso y mantener el workflow durable	orchestrator, daprd	No ejecuta el trabajo ni tiene la clave del modelo
3 · Agentes y apps	Ejecutar el trabajo de cada rol y alojar las aplicaciones de estudiantes	researcher, analyst, executor; Ejercicios	Los agentes no se llaman entre sí; los ejercicios no acceden a Internet
4 · Modelo	Guardar la clave, aplicar presupuesto y elegir modelo real o simulado	llm-gateway, fake-llm	No decide qué hacer: solo controla y reenvía
5 · Mensajes y estado	Entregar eventos y guardar el estado de cada workflow y el presupuesto	NATS JetStream, Redis	No contiene lógica de negocio
6 · Plataforma	Desplegar desde Git, escalar pods y nodos, sincronizar secretos y enviar telemetría	Argo CD, KEDA, Cluster Autoscaler, AWS CCM, External Secrets, Dapr, Alloy	No se cambia a mano: Argo CD revierte los desvíos
7 · Infraestructura	Crear y destruir cómputo, red, secretos y estado	Terraform, EC2, K3s, ASG, SSM, S3	No conoce las aplicaciones

Nivel 3 · Componentes del orquestador

Cada servicio sigue arquitectura hexagonal: las dependencias apuntan hacia el dominio, y 27 contratos de import-linter hacen fallar el CI si alguien las rompe.



Modelo y observabilidad

El modelo real es **NVIDIA nemotron-3.5-bolt-e4b**, detrás de un gateway con API compatible con OpenAI. Sin token de demo válido responde **fake-llm**. **Grafana Cloud** recibe métricas y logs en 6 dashboards por Golden Signals, y **LangSmith**, cada llamada al modelo.

Ciclo de entrega

Un push a **main** ejecuta CI y escaneo de infraestructura, construye 6 imágenes en GHCR y escribe el nuevo tag en **gitops/**. Argo CD lo aplica, informa a GitHub por una GitHub App, y **post-deploy** comprueba el servicio desde fuera, a través del NLB.

Estado de validación

Solo se considera probado lo que tiene evidencia real en el registro de validación y la bitácora.

Área	Estado	Evidencia
Tests y arquitectura de código	**Probado**	Cobertura ≥ 80 % por servicio; 91,2 % en SonarCloud; 27 contratos de arquitectura
Pipeline main	**Probado**	CI, escaneo IaC, 6 imágenes y promoción GitOps
Despliegue desde cero	**Probado**	22 aplicaciones Synced y Healthy, sin pasos manuales
Tarea completa por el NLB	**Probado**	Orquestador y 3 agentes en unos 23 s; KEDA escala desde cero
Autoscaling de nodos	**Probado**	ASG de 1 a 2, nodo Ready con providerID, 2 destinos healthy; bajada automática
GitOps y self-healing	**Probado**	Pod restaurado y cambio manual revertido, 7 s cada uno
Prueba de carga k6	**Probado**	4200 peticiones, 0 % de errores HTTP, p95 de 1,73 s y 0 tokens reales; dejó el cluster saturado al terminar
Ataque	**Probado**	Ráfaga manual de 300: 184 aceptadas y 116 rechazadas con 429; el token falso queda simulado
Modelo real	**Probado**	Tareas completadas con OpenAI y con NVIDIA, solo por configuración
Telemetría	**Probado**	6 dashboards con datos, logs de cada tarea en Loki, métricas de CloudWatch y trazas LLM en LangSmith
Ejercicios de estudiantes	**Probado**	De Pull Request a /s/prueba/ respondiendo 200 por Argo CD
Escritorio remoto	**Probado**	VM por pipeline; Argo CD, Dapr Dashboard y el ejercicio visibles desde ella
Perfil local k3d	**Sin validar**	Descartado por recursos del equipo; la validación se hizo en AWS
Trazas en Grafana Tempo	**Sin validar**	Se envían por Alloy; falta abrir una en Explore

Escenarios de demostración

#	Escenario	Resultado	#	Escenario	Resultado
1	Despliegue	**Probado**	9	Registro en NLB	**Probado**
2	Sync GitOps	**Probado**	10	Distribución	**Corregido**, sin repetir
3	Self-healing	**Probado**	11	Scale-down	**Probado**, observado en AWS
4	Tráfico k6	**Probado** con reservas	12	Grafana	**Probado**, 6 dashboards
5	HPA	**Probado**	13	Ataque	**Probado**, ráfaga manual
6	Pending	**Probado**	14	Chaos Engineering	**Fuera de alcance**
7	Cluster Autoscaler	**Probado**	15	Modelo real	**Probado**
8	Join de K3s	**Probado**			

Enlaces a lo implementado

Requieren iniciar sesión en cada servicio. Argo CD, Dapr y la API del laboratorio solo existen mientras está encendido, y la URL del NLB cambia en cada despliegue.

GitHub

Recurso	Enlace
Repositorio y documentación	Repositorio · README · Guía de presentación
Código	Aplicaciones y agentes · Terraform · GitOps · Aplicaciones de Argo CD
Pipelines	Actions · lab · ui-desktop · main · exercises · lab-reaper
Configuración	Environments · Variables · Secretos
Imágenes (GHCR)	Paquetes del proyecto
Ejercicios	Rama stage · Guía · Ejemplo en Go · PR de prueba
GitHub App	Configuración de la App · Instalaciones

AWS (cuenta 972588015487, región us-east-1)

Recurso	Enlace
Cómputo	Instancias EC2 · Auto Scaling Groups · Load Balancers · Target Groups
Secretos y estado	SSM Parameter Store · Bucket de Terraform
Acceso	CloudShell · Fleet Manager
Identidad y coste	Roles IAM · Budgets · Cost Explorer

Observabilidad y servicios externos

Recurso	Enlace
Grafana Cloud	Instancia · Dashboards · Portal · Service accounts
LangSmith	Trazas de agentic-platform
Modelo real	NVIDIA build
Calidad	Proyecto en SonarCloud · Organización

Disponibles con el laboratorio encendido

Recurso	Dirección	Acceso
API de tareas	http://<dirección-del-NLB>/tasks	Pública
Ejercicios	http://<dirección-del-NLB>/s/<nombre>/	Pública
Argo CD	http://10.0.1.10:30081	Solo dentro de la VPC, desde el escritorio remoto
Dapr Dashboard	http://10.0.1.10:30082	Solo dentro de la VPC, desde el escritorio remoto

La contraseña del escritorio remoto la genera la propia VM y se guarda en el parámetro SSM **/agentic-platform/cluster/ui-desktop-password**. Este informe no incluye ningún valor secreto.

Documentación del proyecto

Documento	Contenido
Guía de presentación	Levantar, demostrar y apagar con un navegador
Guía de configuración	Cuentas, tokens y permisos, paso a paso
Bitácora de despliegue	Qué se hizo, y cada incidencia con su solución
Registro de validación	Evidencia de cada prueba
Plan de implementación	Diseño completo y diagramas
ADR-0001 · ADR-0002	Decisiones de arquitectura y su justificación
Uso en vivo · Runbook	Seguimiento y diagnóstico
Contratos · Versiones	Interfaces entre servicios y versiones fijadas

Incidencias y aprendizajes

La validación en AWS reveló 16 incidencias, todas resueltas o documentadas. Las más relevantes muestran por qué la prueba real era imprescindible.

Problema	Causa	Solución
Promoción GitOps corrupta	Un yq con varios archivos los fusionaba en el primero	Un yq por archivo
GitHub no podía asumir roles de AWS	GitHub emite ahora el subject OIDC en formato inmutable	Confianza actualizada al formato real
Volúmenes sin crear	El taint del control plane expulsaba al pod auxiliar	Toleration aplicada al arrancar el nodo
Sidecars Dapr caídos	El almacén de secretos exige un token no montado	Desactivar ese almacén; los pods siguen sin token
Orquestador sin memoria bajo carga	El sidecar se quedaba corto al reconstruir el historial	Límite del sidecar a 512 Mi y executor a 4 réplicas
Ejercicios sin desplegar	Argo CD no admite dos ramas del mismo repositorio	Chart y values desde la misma rama
Trazas descartadas en LangSmith	Aceptaba con 200 los spans cuyo padre vive en Tempo	Cada llamada se envía como traza raíz

Seguridad, costes y operación

Seguridad

- Sin credenciales de larga duración en CI: GitHub entra en AWS por OIDC, con un rol por entorno.
- Sin SSH ni API de Kubernetes expuestos: el acceso es por Systems Manager.
- Los secretos de ejecución están solo en SSM, cifrados, y se sincronizan con External Secrets. Ningún secreto entra en Git: gitleaks revisa todo el historial en cada pipeline.
- Contenedores distroless sin root, NetworkPolicy default-deny, mTLS entre agentes y aplicaciones de estudiantes sin salida a Internet.

Costes

Estimación con precios on-demand de us-east-1, aún sin contrastar con una factura.

Concepto	Estimación
Laboratorio encendido	Unos 0,20 USD por hora
Noche completa antes de la presentación, 14 h	Unos 2,50 USD
Capa persistente: estado, secretos y roles	Menos de 0,05 USD al mes
GitHub Actions, GHCR y Grafana Cloud	0, en planes gratuitos
Carga y ataque con fake-llm	0
Modelo real, con el endpoint gratuito de NVIDIA	0

Operación y próximos pasos

- **Levantar:** en GitHub Actions, **lab** con **up**, con dos aprobaciones; después, **ui-desktop** para las interfaces.
- **Antes de dejarlo encendido de noche:** poner **LAB_TTL_HOURS** en 18, para que el apagado automático no lo destruya.
- **Apagar: lab** con **down**. Destruye también el escritorio y comprueba que no quede nada facturable.
- **Pendiente:** repetir el escenario 10 sin saturación y abrir una traza en Tempo.
- **Evolución:** Chaos Engineering de caja negra con una herramienta CNCF, como LitmusChaos o Chaos Mesh.